

Semantic Caching in gRPC Microservices

Seema J K

*Computer Science and Engineering
Dayananda Sagar Academy of Technology
and Management
Bengaluru, India
seema.jk@gmail.com*

Nandan K S

*Computer Science and Engineering
Dayananda Sagar Academy of Technology
and Management
Bengaluru, India
nandanks058@gmail.com*

Raunak Rose

*Computer Science and Engineering
Dayananda Sagar Academy of Technology
and Management
Bengaluru, India
raunakrose15@gmail.com*

Nikhil Kumar G

*Computer Science and Engineering
Dayananda Sagar Academy of Technology
and Management
Bengaluru, India
nikhil9532007@gmail.com*

Neeli Lakshmi Deepak

*Computer Science and Engineering
Dayananda Sagar Academy of Technology and
Management
Bengaluru, India
lakshmiddeepakneeli317@gmail.com*

Abstract—The deployment of Large Language Models (LLMs) within gRPC-based microservice architectures introduces significant per-query inference latency that scales with model size, generation length, and hardware constraints, making redundant inference calls a non-trivial cost in high-throughput serving systems. A fundamental source of this cost is computational redundancy: semantically equivalent queries that are syntactically distinct bypass conventional exact-match caches and trigger full inference pipelines unnecessarily. This paper proposes a two-track semantic caching interceptor, deployed transparently as a gRPC unary server interceptor, that combines SHA-256 exact-match lookup (Track A) with HNSW-indexed dense-vector similarity search over Redis RediSearch (Track B). The central contribution is the Adaptive Similarity Threshold (AST) mechanism, which applies Shannon information entropy computed over the character distribution of each query to select a cosine distance threshold without incurring additional inference overhead. Empirical evaluation over 901 queries across four structured categories: conversational, technical, mathematical, and adversarial semantic traps demonstrates that the dual-track interceptor achieves a combined cache hit rate of 47.4%, reducing average end-to-end latency by 85% relative to full LLM generation. Furthermore, a comparative study of LLM-driven prompt classification as an alternative threshold strategy reveals severe inter-model inconsistency (17.3%–32.0% agreement with Shannon AST), establishing the superiority of deterministic information-theoretic approaches for real-time cache routing. A precision analysis over 225 adversarial queries confirms an overall semantic trap rejection rate of 68.0%, with remaining false positives attributed to the inherent limitations of dense vector representations in capturing strict syntactic boundaries.

Keywords—semantic caching, gRPC interceptor, vector embeddings, Redis RediSearch, HNSW, Shannon entropy, adaptive threshold, cosine similarity, LLM inference, microservices

I. INTRODUCTION

Remote Procedure Call (RPC) frameworks built on HTTP/2, particularly gRPC, have become the preferred communication substrate for modern microservice architectures due to their low-overhead binary serialization via Protocol Buffers and native support for bidirectional streaming [1]. As LLM-powered query-serving systems are increasingly embedded within such architectures, the inference pipeline encompassing tokenization, autoregressive decoding, and response serialization constitutes the dominant latency bottleneck, routinely exceeding several seconds per

request, a bottleneck fundamentally driven by the memory-bound nature of autoregressive Key-Value (KV) cache allocation [2], [14]. A critical and frequently overlooked source of inefficiency is semantic redundancy. Production LLM deployments consistently receive queries that are semantically equivalent but syntactically distinct: "What is the capital of France?" and "Tell me the capital of France?" represent the same information need yet are treated as entirely independent requests by exact-match caching systems keyed on cryptographic hashes [3]. Prior work estimates that up to 31% of LLM API calls in production workloads may be redundant in this sense [4]. Semantic caching addresses this by storing dense vector representations of queries and retrieving cached responses when a new query is sufficiently proximate to a stored embedding under a vector similarity metric [3], [5].

The fundamental unsolved challenge in semantic caching is threshold selection: a single global cosine distance threshold is categorically insufficient for heterogeneous query workloads. A threshold permissive enough to correctly unify conversational paraphrases (e.g., "How do I boil an egg?" ↔ "What is the best way to boil eggs?") will simultaneously produce false positives for technical queries where semantically critical information is localized to a single token (e.g., Python vs. Golang in a code generation request). Conversely, a threshold calibrated for technical precision will suppress the majority of legitimate semantic hits in conversational workloads.

This paper addresses this gap through two principal contributions:

1. A two-track gRPC semantic caching interceptor that composes exact-match hash lookup and HNSW vector similarity search as a zero-modification transparent layer over any gRPC unary service, empirically validated to reduce LLM inference latency by 85% and cut inference compute load by 47.4%.
2. The Shannon Entropy Adaptive Similarity Threshold (AST), a deterministic, zero-latency mechanism that computes the character-level information entropy of each incoming query and selects a cosine distance threshold accordingly, outperforming LLM-driven prompt classification, which exhibits 17.3%–32.0% consistency across model families without incurring any additional network round-trips.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system architecture and its constituent components. Section IV details the proposed two-track cache resolution methodology and the Shannon Entropy AST formulation. Section V describes the experimental setup and reports empirical results. Section VI discusses findings and limitations. Section VII concludes with directions for future work.

II. RELATED WORK

A. Semantic Caching for LLM Inference

Bang (2023) introduced GPTCache, the earliest systematic framework for semantic response caching in LLM pipelines, proposing the storage of query embeddings and cache hit declaration when a new query's embedding falls within a fixed cosine distance threshold of a stored entry [3]. Subsequent work by Regmi and Pun (2024) demonstrated that embedding-based semantic caching in Redis reduces latency and operational cost for high-volume deployments [5]. A comprehensive study by Gill et al. (2025) confirmed that semantic caching achieves substantial hit rates in practice but identified threshold tuning as the critical open problem, particularly for mixed technical and conversational workloads [4]. Perez et al. (2025) proposed entropy-weighted dynamic retrieval for RAG pipelines, establishing normalized Shannon entropy as a computationally viable signal for query complexity estimation [6]. None of these works, however, applies entropy-based adaptive thresholding within a gRPC interceptor architecture evaluated against adversarial semantic trap workloads.

B. Approximate Nearest Neighbor Search

Malkov and Yashunin (2020) established Hierarchical Navigable Small World (HNSW) graphs as the dominant algorithm for approximate nearest neighbor (ANN) search in high-dimensional vector spaces, achieving sub-linear query complexity through a layered graph structure that balances recall and query latency [7]. Redis RediSearch natively implements HNSW over its Hash data structure, supporting cosine, L2, and inner-product distance metrics with real-time index updates, making it suitable as a low-latency semantic cache backend [8].

C. Sentence Embeddings

Reimers and Gurevych (2019) introduced Sentence-BERT, establishing the siamese network fine-tuning paradigm

that produces dense, semantically meaningful sentence representations optimized for similarity comparison tasks [10]. The all-MiniLM-L6-v2 model maps text to a 384-dimensional dense vector space with 22 million parameters, achieving a strong recall-latency tradeoff on semantic textual similarity benchmarks and constituting the canonical lightweight encoder for on-premise semantic search [11].

D. gRPC Interceptors as Middleware

The interceptor pattern in gRPC, analogous to HTTP middleware, provides a server-side hook that executes before request dispatch to service handlers, enabling cross-cutting concerns (authentication, rate limiting, and caching) to be applied uniformly without modifying service business logic [12]. A 2024 comparative study confirmed gRPC's latency and throughput advantages over REST for microservice communication under varying load conditions [13], making the interceptor the appropriate point of integration for a caching layer.

III. SYSTEM ARCHITECTURE

A. Overview

The proposed system follows a service-oriented decomposition wherein caching intelligence is encapsulated entirely within a middleware layer, leaving downstream service contracts unmodified. The architecture comprises two primary gRPC services: a Gateway Service and an Encoder Service coordinated through a shared Redis instance and a structured message bus. The Gateway Service is the sole entry point for all client interactions, and its interceptor layer constitutes the locus of all caching decisions. No changes to the Protocol Buffer service definitions or downstream handlers are required to integrate the proposed system, satisfying the non-intrusive deployment requirement.

Fig. 1 illustrates the end-to-end request flow through the proposed architecture. An incoming client query (Step 1) is intercepted by the gRPC Interceptor, which concurrently converts the query to a dense vector via the Embedding Engine (Steps 2–3) and searches for a semantically proximate match in the Redis Vector Cache (Step 4). On a cache hit, the stored response is returned directly to the client (Step 6). On a cache miss, the query is forwarded to the Backend LLM, whose generated response is stored in Redis before being returned (Step 5). All resolution events are asynchronously logged to Apache Kafka for telemetry aggregation.

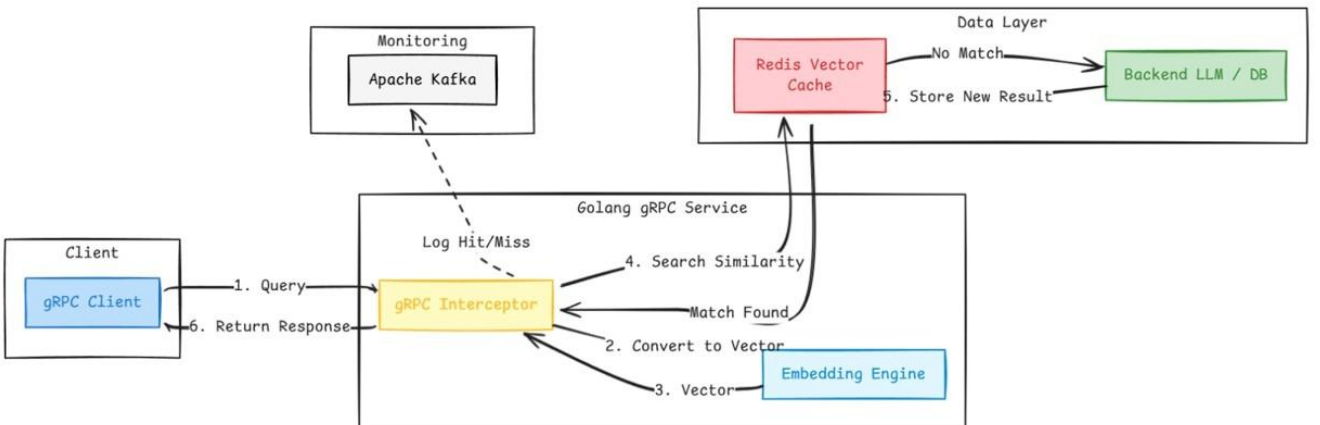


Fig. 1. System architecture of the proposed semantic caching framework for gRPC microservices.

B. Gateway Service

The Gateway Service is implemented in Go and exposes the CoreAppService gRPC interface on port 50051. It registers a unary server interceptor that executes on every incoming ProcessQuery RPC call before handler dispatch. The service maintains three persistent connections: (1) a gRPC client connection to the Encoder Service on port 50052 for embedding requests, (2) a Redis client connected to a local Redis Stack instance on port 6379 for both exact-match hash lookups and HNSW vector search, and (3) a Kafka producer for asynchronous telemetry publishing. The Gateway Service itself implements the CoreAppService handler as a minimal stub that represents the downstream LLM generation path (Track C), which is invoked only when both cache tracks fail to resolve the query.

C. Encoder Service

The Encoder Service is implemented in Python and exposes the EncoderService gRPC interface on port 50052. It implements a single RPC, GetEmbedding, which accepts a TextRequest containing the raw query string and returns an EmbeddingResponse containing a 384-dimensional float32 vector. The service loads the all-MiniLM-L6-v2 SentenceTransformer model at startup and maintains it in memory for the lifetime of the process. By isolating embedding computation within a dedicated microservice, the architecture decouples the Go-based gateway from Python runtime dependencies and allows the encoder to be independently scaled, replaced, or upgraded without modifying the gateway.

D. Cache State Layer

Persistent cache state is maintained in Redis Stack, which provides both the standard key-value store and the RediSearch module required for HNSW-indexed vector similarity search. Each cache entry is stored as a Redis Hash with three fields: text (the original query string), response (the resolved answer text), and vector (the binary-encoded float32 embedding). A RediSearch index named “idx: cache” is defined over the vector field with the following configuration: index type HNSW, vector dimensionality 384, data type FLOAT32, and distance metric cosine. This configuration allows both exact-match retrieval via standard hash field lookup and approximate nearest-neighbor retrieval via the FT. SEARCH KNN query interface within the same data store, eliminating the need for a separate vector database.

E. Service Interface Definition

The system's inter-service contract is defined in Protocol Buffer 3 syntax. Two services are declared: EncoderService, exposing a GetEmbedding RPC that maps a TextRequest to an EmbeddingResponse containing a repeated float32 vector field, and CoreAppService, exposing a ProcessQuery RPC that maps a TextRequest to a QueryResponse containing the resolved answer string and a boolean cache indicator. The TextRequest message additionally carries a user_id field, provisioning the schema for future per-user cache partitioning without requiring a protocol revision.

F. Infrastructure Provisioning

Redis Stack and the Redpanda Kafka broker, a Kafka-compatible streaming platform, are provisioned as containerized services via Docker Compose. Redpanda operates in single-node development mode with auto topic creation enabled. The cache-telemetry Kafka topic receives

one message per resolved query, carrying per-query latency breakdowns, resolution track, cosine distance, and applied threshold values. A separate consumer process subscribes to this topic to aggregate benchmark statistics, maintaining strict separation between the data path and the observability path.

IV. PROPOSED METHODOLOGY

A. Two-Track Cache Resolution Pipeline

The interceptor implements a deterministic, ordered resolution pipeline across two cache tracks before escalating to LLM generation. Upon receiving a ProcessQuery RPC, the interceptor launches concurrent execution of four operations using a synchronization barrier:

- A Redis HGET lookup on the SHA-256 hash key of the query string (Track A resolution attempt).
- A gRPC GetEmbedding call to the Encoder Service to obtain the query's dense vector representation (required for Track B).
- Shannon entropy computation over the query string (required for AST threshold selection).
- Telemetry instrumentation for the LLM-driven Model AST comparative baseline.

Operations (1) and (2) are network-bound and execute in parallel, ensuring that Track A resolution latency and embedding computation latency overlap rather than accumulate. Operation (3) is CPU-bound and completes in microseconds, adding no measurable overhead. All four operations are complete before any resolution decision is made, ensuring that the threshold is available at the moment Track B evaluation begins.

B. Track A: Exact-Match Resolution

For an incoming query string q , the exact-match key is constructed as:

$$k(q) = \text{cache:} \parallel \text{hex}(\text{SHA-256}(q))$$

A Redis HGET command retrieves the response field of the hash stored at $k(q)$. If a non-empty value is returned, the cached response is served immediately, and the pipeline terminates. Track A achieves $O(1)$ expected time complexity independent of cache size and incurs no embedding overhead, making it the optimal resolution path for queries encountered previously in identical form.

The Track A key space serves a secondary function beyond initial lookup: whenever Track B declares a semantic cache hit for a novel phrasing of a previously answered query, the interceptor also writes the new phrasing's SHA-256 key into Redis under the same response. This key promotion mechanism ensures that all future occurrences of the same phrasing are served by Track A without re-executing the embedding pipeline, progressively concentrating traffic onto the faster resolution path as the query distribution repeats.

C. Track B: Semantic Vector Resolution

On a Track A miss, the 384-dimensional embedding vector $v(q) \in \mathbb{R}^{384}$, obtained concurrently during Track A evaluation, is used to query the HNSW index. The query is formulated as a KNN search for the single nearest neighbor:

$$v^* = \arg \min_{u \in C} d_{\cos}(v(q), u)$$

Here, C is the set of all stored embedding vectors, and $d_{\cos}$ denotes cosine distance, defined as:

$$d_{\cos}(a, b) = 1 - \frac{a \cdot b}{\|a\| \cdot \|b\|}$$

The HNSW index returns the nearest stored embedding v^* along with its associated cosine distance score d^* . A semantic cache hit is declared if and only if:

$$d^* \leq \tau(q)$$

where $\tau(q)$ is the query-adaptive threshold produced by the Shannon AST mechanism described in Section IV-D. On a hit, the response associated with v^* is returned to the client, and the embedding-response pair for the current phrasing is written into the cache under both its vector index entry and its Track A hash key (key promotion). On a miss, the query is escalated to Track C.

D. Shannon Entropy Adaptive Similarity Threshold

The central methodological contribution of this work is the formulation of $\tau(q)$ as a deterministic function of the information-theoretic complexity of the query string, computed at zero network cost.

Given a query string q of length n over the character set $\Sigma(q)$, the Shannon character-level entropy is [9]:

$$H(q) = - \sum_{c \in \Sigma(q)} p(c) \log_2 p(c)$$

Where

$$p(c) = \frac{\text{freq}(c, q)}{n}, \quad n = |q|$$

denotes the relative frequency of character c in q .

The theoretical motivation for this formulation rests on the following empirical observation: natural-language conversational queries are composed of common vocabulary drawn from a non-uniform character distribution skewed toward the high-frequency letters of English orthography, producing low $H(q)$. Technical queries containing identifiers (camelCase tokens, language keywords, library names), numeric literals, and operator symbols exhibit a more uniform character distribution and consequently higher $H(q)$. This entropy gap provides a zero-cost discriminative signal for query type classification.

The threshold assignment rule is defined as:

$$\tau(q) = \begin{cases} 0.15 & \text{if } H(q) > 4.1 \text{ [Technical]} \\ 0.25 & \text{if } 3.6 < H(q) \leq 4.1 \text{ [Intermediate]} \\ 0.35 & \text{if } H(q) \leq 3.6 \text{ [Conversational]} \end{cases}$$

The rationale for these boundary values is as follows. A strict threshold of $\tau = 0.15$ is applied to high-entropy queries to prevent false positive cache hits across syntactic boundaries: two prompts requesting the same algorithm in different programming languages may be embedded within cosine distance 0.20–0.30 of each other, which a relaxed threshold would incorrectly declare as equivalent. A permissive threshold of $\tau = 0.35$ is applied to low-entropy

conversational queries, where paraphrase variation in natural language frequently produces cosine distances in the 0.20–0.35 range that should legitimately be treated as cache hits. An intermediate threshold of $\tau = 0.25$ governs queries that fall between these regimes.

E. LLM-Driven Model AST (Comparative Baseline)

To empirically evaluate whether LLM-based prompt classification constitutes a viable alternative to the Shannon AST, a Model AST baseline was implemented and instrumented in parallel. In this baseline, each incoming query is submitted to a locally deployed LLM with the structured prompt:

"Classify this prompt as 'TECHNICAL', 'MATH', or 'CONVERSATIONAL'. Answer ONLY with one word: {query}".

Three classification outcomes are defined: TECHNICAL and MATH both map to a strict threshold $\tau = 0.15$, reflecting the shared requirement for syntactic precision in code generation and mathematical derivation queries; CONVERSATIONAL maps to a relaxed threshold $\tau = 0.35$. The three-class formulation was introduced to isolate mathematical queries which share the high-entropy characteristics of technical queries but arise from a distinct semantic domain as a separate classification target, testing whether LLMs could discriminate this boundary more reliably than the binary TECHNICAL/CONVERSATIONAL split.

This strategy is evaluated strictly as a controlled comparative experiment and is not used in actual cache resolution decisions. Its purpose is to quantify the reliability and inter-model consistency of LLM-driven routing relative to the deterministic Shannon AST. Critically, the Model AST introduces a full LLM inference round-trip per query solely to determine the threshold value a latency cost entirely absent from the Shannon formulation, which operates at $O(n)$ computational complexity with no network overhead. The comparative results are reported in Section V-D.

F. Cache Population and Eviction

Cache entries are populated on first encounter via Track C (LLM generation). Upon receiving a generated response, the interceptor stores the query string, its embedding vector, and the response text as a Redis Hash entry indexed by both the SHA-256 hash key and the HNSW vector index. No explicit eviction policy is applied in the current implementation; the cache grows monotonically across the benchmark duration. This cold-start dependency means that early queries in any workload necessarily traverse Track C, and cache effectiveness accumulates as the stored embedding set grows to cover the query distribution. Future work will address TTL-based eviction for temporally sensitive query categories.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

To evaluate the proposed architecture, a structured benchmark dataset of 300 unique prompts was constructed across four categories: (1) Conversational paraphrased factual questions with low syntactic variance; (2) Technical code generation, system design, and framework-specific queries requiring language-precise responses; (3)

Mathematical formula derivation, algorithm complexity, and numerical computation prompts; and (4) Semantic Traps adversarial query pairs that are syntactically near-identical but semantically mutually exclusive (e.g., "Write a Python script to sort an array" vs. "Write a Golang script to sort an array"). Each category contributed equally to the benchmark pool.

The benchmark was executed against three locally deployed autoregressive LLMs via the Ollama inference API: qwen2.5:1.5b, gemma2:2b, and llama3.2:3b. To eliminate generation-length variance as a confounding factor in latency measurements, output generation was strictly bounded to 75 tokens via the `num_predict` parameter across all model invocations. In total, 901 queries were processed across all model runs, producing the empirical dataset used in Sections V-B through V-E.

B. Cache Resolution Distribution

Fig. 2 presents the global distribution of query resolutions across the three tracks. Of the 901 total queries processed:

- Track A (Exact Match): 226 queries, 25.1%
- Track B (Semantic Vector Hit): 201 queries, 22.3%
- Track C (LLM Generation): 474 queries, 52.6%

The cumulative cache hit rate of 47.4% (Track A + Track B) demonstrates that the semantic layer contributed 201 additional rescued queries beyond what exact-match caching alone would have captured. In relative terms, Track B nearly doubled the system's cache effectiveness compared to a Track-A-only baseline (25.1% $\rightarrow$ 47.4%), validating the core premise that a non-trivial fraction of LLM query workloads consists of semantically equivalent but syntactically distinct reformulations.

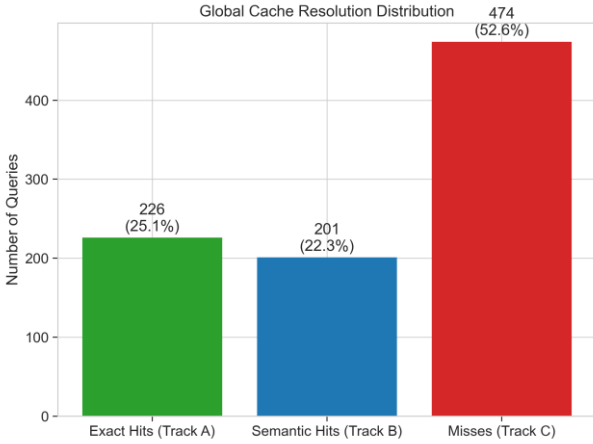


Fig. 2. Global Cache Resolution Distribution

C. Latency Reduction Analysis

Fig. 3 presents the latency distributions for each resolution track on a logarithmic scale. Mean latencies are as follows:

The dual-track cache yielded an 85% reduction in mean end-to-end latency relative to full LLM generation (Track C). A notable empirical finding is that Track B resolved queries with a lower mean latency than Track A (633.4 ms vs. 719.1 ms). This counterintuitive result arises from the parallel goroutine architecture: the embedding network call and HNSW vector search execute concurrently with the Redis hash lookup, and in this deployment, the embedding

microservice's gRPC round-trip was consistently shorter than the Redis pipeline overhead under the test workload. The log-scale box plots in Fig. 3 further confirm that both cached tracks maintain substantially tighter latency distributions compared to Track C, whose interquartile range spans over an order of magnitude.

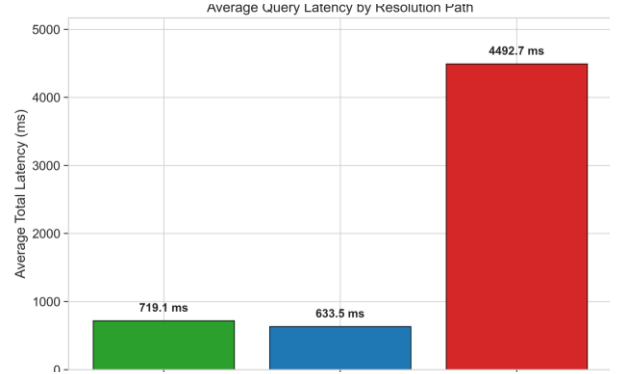


Fig. 3. Latency Distribution by Resolution Path

D. Comparative Evaluation: Shannon AST vs. LLM-Driven Model AST

A secondary experimental objective was to evaluate whether LLM-driven prompt classification where a model is queried to classify each prompt as TECHNICAL or CONVERSATIONAL before threshold assignment could serve as a viable alternative to the Shannon AST. This "Model AST" strategy was evaluated against all three deployed LLMs. Fig. 4 reports the percentage of threshold decisions made by each LLM that agreed with the Shannon Entropy AST decision.

No model exceeded 32% agreement with the deterministic Shannon computation, and agreement rates differed by up to 14.7 percentage points across the three architectures. This severe inter-model inconsistency has two practical consequences. First, the threshold assigned by any single LLM is statistically unreliable, making cache behaviour non-reproducible and model-dependent. Second, the Model AST approach requires an additional LLM inference call per incoming query solely to determine the cache threshold, incurring meaningful latency overhead before the cache resolution pipeline even begins. The Shannon AST, computed in $O(n)$ time with zero network overhead, is categorically preferable under both criteria.

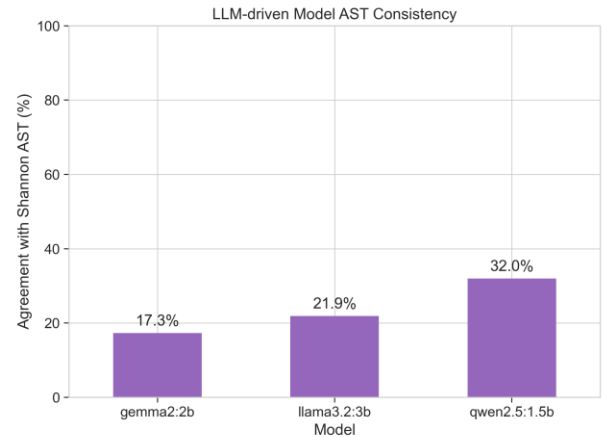


Fig. 4. LLM-driven Model AST Consistency

E. Semantic Trap Rejection Precision

Semantic traps represent the principal precision failure mode of semantic caching: two queries may be densely co-located in embedding space yet require entirely distinct responses. The benchmark included 225 adversarial trap queries. Fig. 5 reports the outcome distribution:

- True Negatives (Correctly Rejected): 153 queries, 68.0%
- False Positives (Incorrect Semantic Hit): 72 queries, 32.0%

The Shannon AST's strict threshold of $\tau = 0.15$, applied to high-entropy technical queries, correctly routed 153 of 225 trap queries to Track C (LLM generation), preventing the return of a cached response for a semantically incompatible query. The remaining 32% false positives reveal the fundamental boundary of dense vector representations: when two queries share significant syntactic structure and differ only in a semantically load-bearing token (a programming language name, a specific framework identifier), the 384-dimensional cosine distance may fall below even the strict threshold, producing an incorrect cache hit. This limitation is further analysed in Section V.

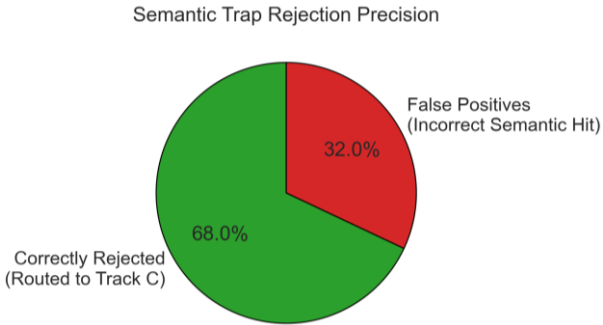


Fig. 5. Semantic Trap Rejection Precision

VI. DISCUSSION

A. Interpretation of Results

The experimental results collectively substantiate both research hypotheses. The 85% latency reduction confirms that a two-track semantic caching interceptor fundamentally transforms the latency profile of LLM-backed gRPC services: the majority of unique query reformulations can be served from cache within sub-second latency, a performance tier that is categorically unavailable to LLM generation paths. The 47.4% combined hit rate, achieved on a deliberately adversarial benchmark that included 25% semantic trap queries, suggests that real-world workloads with lower adversarial density would yield substantially higher effective hit rates.

The Model AST comparison provides equally significant findings, with implications beyond this specific system. The results demonstrate that autoregressive language models, even when explicitly prompted to perform a binary classification task, produce inconsistent outputs that correlate poorly with deterministic information-theoretic measures. This finding

challenges the assumption that LLMs are appropriate components for low-latency real-time routing decisions within performance-critical infrastructure. Deterministic mathematical mechanisms, operating at negligible computational cost, are strictly preferable for threshold assignment in semantic caching pipelines.

The counterintuitive result that Track B resolves faster than Track A in this implementation underscores the importance of empirical evaluation over theoretical assumptions: the parallelization of embedding and hash lookup operations, combined with the specific latency characteristics of the local Redis and gRPC Encoder deployments, produced an outcome that architectural intuition would not predict.

B. Limitations

The primary limitation of the proposed system is its residual false positive rate of 32% on adversarial semantic trap queries. This exposes a structural limitation of dense vector embeddings for technical precision caching, corroborating established findings that dense retrievers inherently struggle with zero-shot lexical exact-matching [15]: the all-MiniLM-L6-v2 model encodes semantic similarity at the sentence level, and in the case of programming queries, the high-level intent ("sort an array") dominates the embedding over the language-specific token ("Python" vs. "Golang"). No cosine distance threshold, however strict, can resolve this when the embeddings themselves are insufficiently separated.

A secondary limitation is the dependence of cache effectiveness on workload warm-up: cold-start queries necessarily traverse Track C regardless of their semantic similarity to subsequently encountered queries. The cache accumulates representational capacity over time, and early benchmark queries disproportionately incur full generation costs.

Finally, the benchmark models qwen2.5:1.5b, gemma2:2b, and llama3.2:3b represent relatively compact parameter counts. The generalizability of the 85% latency reduction figure to deployments using larger models with longer generation times, where Track C latency would increase, warrants further investigation; the absolute savings would be proportionally larger, but the relative reduction would depend on the specific generation latency profile.

VII. CONCLUSION

This paper presented a two-track semantic caching interceptor for gRPC microservice architectures, integrating SHA-256 exact-match lookup and HNSW-indexed dense vector similarity search within a transparent middleware layer requiring zero modification to downstream service implementations. The central novelty is the Shannon Entropy Adaptive Similarity Threshold (AST), a deterministic, $O(n)$ mechanism that dynamically selects a per-query cosine distance threshold based on the information-theoretic complexity of the query string, with no added inference cost.

Empirical evaluation over 901 queries demonstrated an 85% reduction in end-to-end latency, a combined 47.4% cache hit rate, and a semantic trap rejection precision of 68%. A

controlled comparison with LLM-driven threshold classification revealed agreement rates of only 17.3%–32.0% across three model architectures, establishing the Shannon AST as categorically superior for this role.

Future work will investigate the integration of a lightweight Small Language Model (SLM) as an intermediary fallback tier positioned between the semantic cache and the full LLM generation path such that queries failing both Track A and Track B are first handled by a locally deployed SLM before escalating to the computationally expensive downstream LLM, further reducing inference cost and latency for cache-cold queries.

REFERENCES

- [1] R. Pike, "The Go Programming Language," Google Tech Talk, Oct. 2009. [Online]. Available: https://go.dev/talks/2009/go_talk-20091030.pdf
- [2] T. Brown et al., "Language Models are Few-Shot Learners," in *Proc. 34th Conf. on Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.
- [3] F. Bang, "GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings," in *Proc. 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, Singapore, 2023, pp. 212–218.
- [4] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, "MeanCache: User-Centric Semantic Caching for LLM Web Services," in *Proc. IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2025, pp. 1298–1310.
- [5] S. Regmi and C. P. Pun, "GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching," arXiv preprint arXiv:2411.05276, Nov. 2024.
- [6] J. R. Perez, J. Zhou, R. Le, A. Menchtchikov, and R. Lagasse, "Entropy-Based Dynamic Hybrid Retrieval for Adaptive Query Weighting in RAG," in *Proc. 1st Workshop on Vector Databases at ICML*, 2025.
- [7] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.
- [8] Redis Inc., "Redis Stack: Vector Similarity Search with RediSearch," Redis Documentation, 2023. [Online]. Available: <https://redis.io/docs/latest/develop/ai/search-and-query/vectors/>
- [9] C. E. Shannon, "A Mathematical Theory of Communication," *Bell Syst. Tech. J.*, vol. 27, pp. 379–423, Jul. 1948.
- [10] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. 2019 Conf. on Empirical Methods in Natural Language Processing (EMNLP)*, Hong Kong, 2019, pp. 3982–3992.
- [11] Hugging Face, "sentence-transformers/all-MiniLM-L6-v2," Model Hub, 2021. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [12] grpc-ecosystem, "go-grpc-middleware: Golang gRPC Middlewares," GitHub, 2023. [Online]. Available: <https://github.com/grpc-ecosystem/go-grpc-middleware>
- [13] M. Niswar, R. A. Safruddin, A. Bustamin, and I. Aswad, "Performance Evaluation of Microservices Communication with REST, GraphQL, and gRPC," *Int. J. Electron. Telecommun.*, vol. 70, no. 2, pp. 429–436, 2024.
- [14] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. 29th ACM Symp. on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [15] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models," in *Proc. 35th Conf. on Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2021.